

Frontend Architecture

Next.js 16.3 App Router, TypeScript strict, Tailwind v4, shadcn/ui, TanStack Query v5.

Next 16 changes that affect this design

Four things differ from most Next.js material you will find, including the App Router tutorials written for 14 and 15:

Change	Consequence
<code>middleware.ts</code> is deprecated and renamed to <code>proxy.ts</code>	The exported function must be named <code>proxy</code> . Node.js runtime only — edge is not supported.
<code>params</code> , <code>searchParams</code> , <code>cookies()</code> , <code>headers()</code> are async-only	The Next 15 sync compatibility shim is gone. Every page that reads <code>params</code> must <code>await</code> them.
<code>next lint</code> was removed; <code>next build</code> no longer lints	ESLint runs as its own script with flat config. The <code>eslint</code> key in <code>next.config</code> is also gone.
Turbopack is the default	Passing <code>--turbo</code> is now redundant.

If you pin to Next 15 instead, rename `proxy.ts` → `middleware.ts` and the exported function `proxy` → `middleware`. Everything else in this document is unchanged.

package.json

Exact versions, no carets. A blueprint an engineer implements verbatim should be reproducible; let Renovate or Dependabot do the bumping later.

```
{
  "name": "ap-invoice-client",
  "version": "0.1.0",
  "private": true,
  "engines": { "node": ">=20.9.0" },
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "typegen": "next typegen",
    "lint": "eslint .",
    "lint:fix": "eslint . --fix",
    "typecheck": "tsc --noEmit",
    "format": "prettier --write \"**/*.ts,tsx,css,json,md\"",
    "format:check": "prettier --check \"**/*.ts,tsx,css,json,md\""
  },
  "dependencies": {
    "next": "16.3.0",
    "react": "19.2.8",
    "react-dom": "19.2.8",

    "@tanstack/react-query": "5.101.4",
    "axios": "1.19.0",

    "react-hook-form": "7.85.0",
    "@hookform/resolvers": "5.7.1",
    "zod": "4.4.3",

    "react-dropzone": "20.1.0",
```

```

"react-pdf": "10.4.1",
"pdfjs-dist": "5.4.296",

"@radix-ui/react-avatar": "1.1.11",
"@radix-ui/react-checkbox": "1.3.3",
"@radix-ui/react-dialog": "1.1.23",
"@radix-ui/react-dropdown-menu": "2.1.18",
"@radix-ui/react-label": "2.1.8",
"@radix-ui/react-popover": "1.1.23",
"@radix-ui/react-scroll-area": "1.2.10",
"@radix-ui/react-select": "2.2.7",
"@radix-ui/react-separator": "1.1.8",
"@radix-ui/react-slot": "1.3.3",
"@radix-ui/react-switch": "1.2.7",
"@radix-ui/react-tabs": "1.1.14",
"@radix-ui/react-tooltip": "1.2.9",

"cmdk": "1.1.1",
"react-resizable-panels": "4.12.2",
"sonner": "2.0.8",
"lucide-react": "1.31.0",
"next-themes": "0.4.6",
"date-fns": "4.4.0",

"class-variance-authority": "0.7.1",
"clsx": "2.1.1",
"tailwind-merge": "3.6.0"
},
"devDependencies": {
  "typescript": "6.0.3",
  "@types/node": "26.2.0",
  "@types/react": "19.2.18",
  "@types/react-dom": "19.2.4",

  "tailwindcss": "4.3.3",
  "@tailwindcss/postcss": "4.3.3",
  "tw-animate-css": "1.4.0",

  "eslint": "10.8.1",
  "eslint-config-next": "16.3.0",
  "typescript-eslint": "8.67.0",

  "prettier": "3.9.6",
  "prettier-plugin-tailwindcss": "0.8.1",

  "@tanstack/react-query-devtools": "5.101.4"
},
"overrides": {
  "pdfjs-dist": "5.4.296"
}
}

```

Two pins are not negotiable:

- `pdfjs-dist` at exactly `5.4.296`. `react-pdf@10.4.1` depends on that exact version. Declaring `^6` hoists 6.x to the top level while `react-pdf` keeps a nested 5.x, so `new URL('pdfjs-dist/build/pdf.worker.min.mjs', import.meta.url)` resolves to the 6.x worker running against the 5.x API. The `overrides` block is belt-and-braces, guaranteeing one copy of `pdfjs` in the tree no matter what a future transitive dependency asks for.
- `TypeScript` at `6.0.3`, not the `latest` `7.0.2`. `typescript-eslint@8.67.0` declares `typescript: ">=4.8.4 <6.1.0"`. TS 7 is the Go port and has no stable programmatic API yet, so linting simply cannot consume it. Revisit only once `typescript-eslint` widens that range.

Directory tree

```
client/
├── .env.example
├── .env.local # gitignored
├── eslint.config.mjs # flat config (Next 16 requirement)
├── next.config.ts
├── postcss.config.mjs
├── tsconfig.json
├── components.json # shadcn config
├── package.json
├── proxy.ts # route protection (was middleware.ts)
└── src/
    ├── app/
    │   ├── layout.tsx # RSC · html/body, fonts, Providers, Toaster
    │   ├── globals.css # Tailwind v4 @import + @theme tokens
    │   ├── error.tsx # 'use client' global error boundary
    │   ├── not-found.tsx # RSC
    │   └── page.tsx # RSC · redirect() to /dashboard or /login
    │   ├── (auth)/
    │   │   ├── layout.tsx # RSC · centered card shell
    │   │   ├── login/page.tsx # RSC shell -> <LoginForm/>
    │   │   └── register/page.tsx # RSC shell -> <RegisterForm/>
    │   ├── (dashboard)/
    │   │   ├── layout.tsx # RSC · fetches /auth/me, renders chrome
    │   │   ├── dashboard/
    │   │   │   ├── page.tsx # RSC · prefetch history -> HydrationBoundary
    │   │   │   ├── loading.tsx # RSC · skeleton
    │   │   │   └── error.tsx # 'use client'
    │   │   ├── upload/page.tsx # RSC shell -> <InvoiceUploader/>
    │   │   ├── verify/[matchId]/
    │   │   │   ├── page.tsx # RSC · await params, prefetch detail
    │   │   │   ├── loading.tsx
    │   │   │   └── error.tsx
    │   │   └── knowledge-base/
    │   │       ├── page.tsx
    │   │       └── loading.tsx
    │   └── api/auth/ # BFF hop: sets httpOnly cookies
    │       ├── login/route.ts
    │       ├── register/route.ts
    │       ├── refresh/route.ts
    │       └── logout/route.ts
    ├── components/
    │   ├── ui/ # shadcn generated - do not hand-edit
    │   ├── layout/
    │   │   ├── app-sidebar.tsx # 'use client' (active-link state)
    │   │   ├── app-topbar.tsx
    │   │   ├── user-menu.tsx
    │   │   └── theme-toggle.tsx
    │   ├── shared/
    │   │   ├── confidence-score-badge.tsx
    │   │   ├── status-badge.tsx
    │   │   ├── empty-state.tsx
    │   │   ├── data-table-pagination.tsx
    │   │   └── currency.tsx
    │   ├── features/
    │   │   ├── auth/
    │   │   │   ├── login-form.tsx
    │   │   │   ├── register-form.tsx
    │   │   │   └── auth-schemas.ts
    │   │   ├── upload/
    │   │   │   ├── invoice-uploader.tsx # orchestrates dropzone + progress
    │   │   │   ├── invoice-dropzone.tsx # react-dropzone
    │   │   │   └── upload-progress-card.tsx
    │   │   ├── dashboard/
    │   │   │   ├── match-history-table.tsx
    │   │   │   ├── match-history-row.tsx
    │   │   │   ├── match-history-filters.tsx # URL searchParams
    │   │   │   └── stats-cards.tsx
    │   │   └── verification/
    │   │       └── verification-screen.tsx # top-level state owner
```

```

├── verification-toolbar.tsx
├── invoice-preview.tsx      # dynamic(ssr:false) switch
├── pdf-viewer.tsx          # react-pdf + worker config
├── image-viewer.tsx
├── viewer-controls.tsx
├── comparison-table.tsx
├── line-items-comparison.tsx
├── po-selector-combobox.tsx
├── score-breakdown-panel.tsx
├── candidate-list.tsx
├── correction-form.tsx
├── confirm-match-dialog.tsx
├── verification-schemas.ts
├── knowledge-base/
│   ├── vendor-alias-table.tsx
│   ├── vendor-alias-dialog.tsx
│   └── alias-schemas.ts
├── hooks/
│   ├── use-invoice.ts
│   ├── use-auth.ts
│   ├── use-knowledge-base.ts
│   └── use-debounced-value.ts
├── lib/
│   ├── api-client.ts        # browser axios instance
│   ├── api-server.ts        # RSC fetch with cookie forwarding
│   ├── query-keys.ts        # centralized key factory
│   ├── errors.ts            # ApiError normalization
│   ├── format.ts            # currency/date formatters
│   ├── confidence.ts         # threshold -> tier mapping
│   ├── compare.ts           # field/line-item diffing
│   └── utils.ts              # cn()
├── providers/
│   ├── index.tsx
│   ├── query-provider.tsx
│   └── theme-provider.tsx
├── types/
│   ├── invoice.ts           # see document 04
│   ├── auth.ts
│   ├── knowledge-base.ts
│   └── api.ts

```

Auth strategy — the decision that shapes everything else

The constraint

`proxy.ts` runs on the server. It cannot read `localStorage`. So if the JWT lives in `localStorage`, `proxy.ts` cannot protect anything — it degrades into a client-side redirect that flashes protected UI before bouncing, which is security theatre rather than security.

There is a second, product-specific constraint most write-ups miss: **the verification screen must render the original PDF**. With a token in `localStorage`, `<Document file="/api/v1/invoices/x/file" />` carries no auth header, so you must `fetch` the file as a blob, `URL.createObjectURL` it, and manually revoke it on unmount. That is real, leak-prone complexity on the single most important screen in the product.

Recommendation: httpOnly cookie + same-origin rewrite

1. The browser posts credentials to a Next **Route Handler** (`/api/auth/login`), not directly to FastAPI.
2. That handler calls FastAPI, receives the JWT pair, and sets them as `httpOnly; Secure; SameSite=Lax` cookies. **The token never touches JavaScript**, so XSS cannot exfiltrate it.

3. `next.config.ts` rewrites `/api/v1/:path*` to FastAPI, making all app traffic same-origin, so the browser attaches the cookie automatically.
4. `proxy.ts` reads the cookie and performs real server-side route protection.
5. FastAPI accepts the JWT from **either** the `Authorization: Bearer` header **or** the `access_token` cookie — a ten-line dependency, already shown in document 01.

	httpOnly cookie + rewrite	localStorage + CORS
XSS token theft	Not possible	Trivially possible
<code>proxy.ts</code> protection	Real	Impossible
PDF <code><Document file=...></code>	Just works	Blob fetch + objectURL lifecycle
CORS preflight	None (same origin)	On every request
RSC prefetching	Forward the cookie via <code>cookies()</code>	Impossible — token is client-only
Backend change needed	Small (cookie-or-header dependency)	None

The only cost is that ten-line FastAPI dependency. Since the backend is greenfield, that cost is essentially zero — take it now, before it gets expensive.

CSRF. `SameSite=Lax` blocks cross-site POSTs, which covers the realistic threat model for an internal AP tool. If this is ever exposed to the public internet, add a double-submit CSRF token in the Route Handlers.

If the backend team refuses cookie auth, fall back to `localStorage` — but then **delete** `proxy.ts` **entirely** rather than shipping a redirect that pretends to protect something.

app/api/auth/login/route.ts

```
import { NextResponse, type NextRequest } from 'next/server';

const API_URL = process.env.API_BASE_URL!; // server-only, no NEXT_PUBLIC_ prefix
const isProd = process.env.NODE_ENV === 'production';

export async function POST(request: NextRequest) {
  const body = await request.json();

  const upstream = await fetch(`${API_URL}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body),
    cache: 'no-store',
  });

  const data = await upstream.json();
  if (!upstream.ok) {
    return NextResponse.json(data, { status: upstream.status });
  }

  // Return the user, but NEVER the tokens - they exist only as httpOnly cookies.
  const response = NextResponse.json({ user: data.user });

  const common = {
    httpOnly: true,
    secure: isProd,
    sameSite: 'lax' as const,
    path: '/',
  };
  response.cookies.set('access_token', data.access_token, {
    ...common,
    maxAge: 60 * 30, // 30 minutes
  });
  response.cookies.set('refresh_token', data.refresh_token, {
    ...common,
    // Scoped so the refresh token is never sent to any other route.
    path: '/api/auth',
    maxAge: 60 * 60 * 24 * 7,
  });

  return response;
}
```

proxy.ts — route protection

```
import { NextResponse, type NextRequest } from 'next/server';

const ACCESS_COOKIE = 'access_token';
const AUTH_ROUTES = ['/login', '/register'];

/**
 * Decodes the JWT payload WITHOUT verifying the signature.
 *
 * This is intentional and safe: proxy is a UX redirect layer, not an
 * authorization boundary. FastAPI verifies the signature on every request.
 * Verifying here would mean shipping the signing secret into the Next runtime
 * for zero security gain. We read only 'exp', so an obviously-dead token
 * redirects immediately instead of rendering a shell that then 401s.
 */
function isExpired(token: string): boolean {
  try {
    const payload = token.split('.')[1];
    if (!payload) return true;
    const json = Buffer.from(payload, 'base64url').toString('utf8');
    const { exp } = JSON.parse(json) as { exp?: number };
    if (typeof exp !== 'number') return false; // no exp claim - let the API decide
    return exp * 1000 <= Date.now();
  } catch {
    return true;
  }
}
```

```

}

export function proxy(request: NextRequest) {
  const { pathname, search } = request.nextUrl;
  const token = request.cookies.get(ACCESS_COOKIE)?.value;
  const isAuthenticated = Boolean(token) && !isExpired(token!);
  const isAuthenticatedRoute = AUTH_ROUTES.some((r) => pathname.startsWith(r));

  // Signed-in users should never see login/register.
  if (isAuthenticatedRoute) {
    if (isAuthenticated) return NextResponse.redirect(new URL('/dashboard', request.url));
    return NextResponse.next();
  }

  if (!isAuthenticated) {
    const login = new URL('/login', request.url);
    login.searchParams.set('next', pathname + search);
    const response = NextResponse.redirect(login);
    // Clear a stale cookie so we don't loop on the next navigation.
    if (token) response.cookies.delete(ACCESS_COOKIE);
    return response;
  }

  return NextResponse.next();
}

export const config = {
  matcher: [
    // Everything except /api/*, Next internals, and static asset extensions.
    '/(?!(?!api|_next/static|_next/image|favicon.ico|robots.txt|sitemap.xml|.*/\\.
    (?:svg|png|jpg|jpeg|gif|webp|ico)$).*')',
  ],
};

```

Never treat `proxy.ts` as your only authorization check. Next's own docs warn that Server Functions POST to the route that uses them, so a matcher change can silently drop proxy coverage. FastAPI must verify every request independently; `proxy.ts` exists purely to avoid rendering protected shells to signed-out users.

lib/api-client.ts

```

import axios, {
  type AxiosError,
  type AxiosInstance,
  type AxiosRequestConfig,
  type InternalAxiosRequestConfig,
} from 'axios';
import type { ApiErrorBody } from '@types/api';

/**
 * Same-origin by default: next.config.ts rewrites /api/v1/* to FastAPI, so the
 * httpOnly auth cookie rides along automatically and there is no CORS.
 */
const BASE_URL = process.env.NEXT_PUBLIC_API_BASE_URL ?? '/api/v1';

export const apiClient: AxiosInstance = axios.create({
  baseURL: BASE_URL,
  // OCR + Odoos matching is genuinely slow. Do not lower this.
  timeout: 120_000,
  withCredentials: true,
  headers: { Accept: 'application/json' },
});

/* -----
 * Request interceptor
 * ----- */

/**
 * In the recommended cookie mode this is a no-op - the browser attaches the

```

```

* httpOnly cookie itself and JS cannot read it. It exists so the localStorage
* fallback is a one-line change rather than a refactor.
*/
function readFallbackToken(): string | null {
  if (process.env.NEXT_PUBLIC_AUTH_MODE !== 'localStorage') return null;
  if (typeof window === 'undefined') return null;
  return window.localStorage.getItem('access_token');
}

apiClient.interceptors.request.use((config: InternalAxiosRequestConfig) => {
  const token = readFallbackToken();
  if (token) config.headers.set('Authorization', `Bearer ${token}`);
  if (config.method?.toLowerCase() === 'get') {
    config.headers.set('Cache-Control', 'no-cache');
  }
  return config;
});

/* -----
* Response interceptor: single-flight refresh, then redirect on failure
* ----- */

interface RetriableConfig extends AxiosRequestConfig {
  _retry?: boolean;
}

/**
* Shared promise, so N concurrent 401s trigger exactly ONE refresh call.
* Without this, a dashboard that fires four parallel queries on an expired
* token fires four refreshes and races itself into a logout loop. This is the
* single most commonly omitted detail in axios auth setups.
*/
let refreshPromise: Promise<boolean> | null = null;

async function refreshSession(): Promise<boolean> {
  try {
    // Hits the Next Route Handler (not FastAPI) so it can rotate the httpOnly
    // cookies. Bare fetch, not apiClient, to avoid recursive interception.
    const res = await fetch('/api/auth/refresh', {
      method: 'POST',
      credentials: 'include',
    });
    return res.ok;
  } catch {
    return false;
  }
}

function redirectToLogin(): void {
  if (typeof window === 'undefined') return;
  const next = encodeURIComponent(
    window.location.pathname + window.location.search,
  );
  if (window.location.pathname !== '/login') {
    window.location.replace(`/login?next=${next}`);
  }
}

apiClient.interceptors.response.use(
  (response) => response,
  (error: AxiosError<ApiErrorBody>) => {
    const status = error.response?.status;
    const original = error.config as
      | (RetriableConfig & InternalAxiosRequestConfig)
      | undefined;

    // Only 401 is recoverable. 403 means authenticated-but-forbidden: never
    // refresh on it, or you mask a permissions bug as a session problem.
    if (status !== 401 || !original || original._retry) {
      return Promise.reject(error);
    }

    // Never try to refresh a failed refresh or login - that is a hard logout.
    const url = original.url ?? '';
    if (url.includes('/auth/refresh') || url.includes('/auth/login')) {
      redirectToLogin();
      return Promise.reject(error);
    }
  }
);

```



```

    }

    original._retry = true;

    refreshPromise ??= refreshSession().finally(() => {
      refreshPromise = null;
    });

    const refreshed = await refreshPromise;
    if (!refreshed) {
      redirectToLogin();
      return Promise.reject(error);
    }

    return apiClient(original);
  },
);

```

lib/errors.ts

FastAPI's `detail` is a union of `string` and `ValidationError[]`. Normalizing it once here means no component ever has to handle both shapes.

```

import axios from 'axios';
import type { ApiErrorBody, FastAPIValidationError } from '@types/api';

export class ApiError extends Error {
  readonly status: number;
  readonly fieldErrors: Record<string, string>;

  constructor(
    message: string,
    status: number,
    fieldErrors: Record<string, string> = {},
  ) {
    super(message);
    this.name = 'ApiError';
    this.status = status;
    this.fieldErrors = fieldErrors;
  }
}

function isValidErrors(d: unknown): d is FastAPIValidationError[] {
  return Array.isArray(d);
}

export function toApiError(error: unknown): ApiError {
  if (!axios.isAxiosError<ApiErrorBody>(error)) {
    return new ApiError(
      error instanceof Error ? error.message : 'Unexpected error',
      0,
    );
  }

  const status = error.response?.status ?? 0;
  const detail = error.response?.data?.detail;

  if (typeof detail === 'string') return new ApiError(detail, status);

  if (isValidErrors(detail)) {
    const fieldErrors: Record<string, string> = {};
    for (const issue of detail) {
      // loc looks like ["body", "vendor_name"] - drop the "body" prefix.
      const field = issue.loc.filter((p) => p !== 'body').join('.');
      if (field) fieldErrors[field] = issue.msg;
    }
    return new ApiError('Validation failed', status, fieldErrors);
  }

  if (error.code === 'ECONNABORTED') {
    return new ApiError(
      'The request timed out. The document may still be processing.',
      408,
    );
  }
}

```

```

    });
  }

  return new ApiError(error.message || 'Request failed', status);
}

```

providers/query-provider.tsx

```

'use client';

import { useState } from 'react';
import {
  QueryClient,
  QueryClientProvider,
  isServer,
  MutationCache,
} from '@tanstack/react-query';
import { ReactQueryDevtools } from '@tanstack/react-query-devtools';
import { toast } from 'sonner';
import { toApiError } from '@lib/errors';

function makeQueryClient(): QueryClient {
  return new QueryClient({
    defaultOptions: {
      queries: {
        // 60s: OCR results are immutable once produced, so aggressive caching is
        // safe and makes back-navigation from verify -> dashboard instant.
        staleTime: 60_000,
        gcTime: 5 * 60_000,
        refetchOnWindowFocus: false,
        retry: (failureCount, error) => {
          const { status } = toApiError(error);
          // Never retry auth/permission/not-found/validation failures.
          if ([400, 401, 403, 404, 422].includes(status)) return false;
          return failureCount < 2;
        },
        retryDelay: (attempt) => Math.min(1000 * 2 ** attempt, 30_000),
      },
      mutations: {
        // Uploads and Odoo pushes are NOT idempotent. A retried confirm could
        // create a second vendor bill.
        retry: false,
      },
    },
    mutationCache: new MutationCache({
      onError: (error) => {
        const { status, message } = toApiError(error);
        if (status === 401) return; // the interceptor already redirects
        toast.error(message);
      },
    }),
  });
}

let browserQueryClient: QueryClient | undefined;

/**
 * Server: always a fresh client, so no state leaks between requests - critical
 * for a multi-tenant SaaS where a cached query could otherwise be served to the
 * wrong organization.
 * Browser: a singleton, created lazily so React 19 Suspense cannot discard it
 * mid-render.
 */
function getQueryClient(): QueryClient {
  if (isServer) return makeQueryClient();
  browserQueryClient ??= makeQueryClient();
  return browserQueryClient;
}

export function QueryProvider({ children }: { children: React.ReactNode }) {
  // useState, not useMemo - guaranteed not to be re-run on re-render.
  const [queryClient] = useState(getQueryClient);

```

```

return (
  <QueryClientProvider client={queryClient}>
    {children}
    {process.env.NODE_ENV === 'development' && (
      <ReactQueryDevtools initialIsOpen={false} buttonPosition="bottom-left" />
    )}
  </QueryClientProvider>
);
}

```

hooks/use-invoice.ts

```

'use client';

import { useCallback, useState } from 'react';
import {
  keepPreviousData,
  useMutation,
  useQuery,
  useQueryClient,
} from '@tanstack/react-query';
import type { AxiosProgressEvent } from 'axios';
import { toast } from 'sonner';
import { apiClient } from '@lib/api-client';
import { queryKeys, type InvoiceListFilters } from '@lib/query-keys';
import { useDebounceValue } from '@hooks/use-debounce-value';
import type { Paginated } from '@types/api';
import type {
  ConfirmMatchRequest, ConfirmMatchResponse, MatchDetail,
  MatchHistory, MatchResult, POCandidate,
} from '@types/invoice';

/* =====
 * List - dashboard match history
 * ===== */

export function useMatchHistory(filters: InvoiceListFilters) {
  return useQuery({
    queryKey: queryKeys.invoices.list(filters),
    queryFn: async ({ signal }) => {
      const { data } = await apiClient.get<Paginated<MatchHistory>>('/invoices', {
        params: {
          page: filters.page,
          page_size: filters.pageSize,
          status: filters.status || undefined,
          q: filters.q || undefined,
        },
        signal,
      });
      return data;
    },
    // Keeps the current page visible while the next loads - no table flicker.
    placeholderData: keepPreviousData,
  });
}

/* =====
 * Upload - with real progress
 * ===== */

export interface UploadProgress {
  loaded: number;
  total: number;
  percent: number;
  /** True once bytes are fully sent and we're waiting on OCR + matching. */
  processing: boolean;
}

const INITIAL_PROGRESS: UploadProgress = {
  loaded: 0, total: 0, percent: 0, processing: false,
};

export function useUploadInvoice() {

```

```

const queryClient = useQueryClient();
const [progress, setProgress] = useState<UploadProgress>(INITIAL_PROGRESS);
const [controller, setController] = useState<AbortController | null>(null);

const mutation = useMutation({
  mutationFn: async (file: File): Promise<MatchResult> => {
    const abort = new AbortController();
    setController(abort);
    setProgress({ ...INITIAL_PROGRESS, total: file.size });

    const formData = new FormData();
    formData.append('file', file);

    const { data } = await apiClient.post<MatchResult>('invoices/upload',
      formData,
      {
        signal: abort.signal,
        // Do NOT set Content-Type manually - the browser must generate the
        // multipart boundary. Setting it by hand is the #1 upload bug.
        onUploadProgress: (event: AxiosProgressEvent) => {
          const total = event.total ?? file.size;
          const percent =
            total > 0 ? Math.round((event.loaded / total) * 100) : 0;
          setProgress({
            loaded: event.loaded,
            total,
            percent,
            // At 100% the bytes are sent but the server is still doing OCR and
            // Odoo matching, which takes 10-30s. Flip to an indeterminate
            // "processing" state so the bar doesn't sit at 100% looking hung.
            processing: percent >= 100,
          });
        },
      },
    );

    return data;
  },
  onSuccess: (result) => {
    // Seed the detail cache so /verify/[matchId] renders with zero fetch.
    queryClient.setQueryData(queryKeys.invoices.detail(result.id), result);
    void queryClient.invalidateQueries({
      queryKey: queryKeys.invoices.lists(),
    });
  },
  onSettled: () => setController(null),
});

const cancel = useCallback(() => {
  controller?.abort();
  setProgress(INITIAL_PROGRESS);
}, [controller]);

return { ...mutation, progress, cancel };
}

/* =====
 * Detail - verification screen
 * ===== */

export function useMatchDetail(matchId: string, enabled = true) {
  return useQuery({
    queryKey: queryKeys.invoices.detail(matchId),
    queryFn: async ({ signal }) => {
      const { data } = await apiClient.get<MatchDetail>(
        `invoices/${matchId}`,
        { signal },
      );
      return data;
    },
    enabled: enabled && Boolean(matchId),
    // A row still being processed polls itself to completion. This is also what
    // makes an async (202 + poll) backend a drop-in change later.
    refetchInterval: (query) => {

```

```

    const status = query.state.data?.status;
    return status === 'pending' || status === 'processing' ? 3000 : false;
  },
});
}

/* =====
 * Confirm
 * ===== */

export function useConfirmMatch(matchId: string) {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: async (
      payload: ConfirmMatchRequest,
    ): Promise<ConfirmMatchResponse> => {
      const { data } = await apiClient.post<ConfirmMatchResponse>(
        `/invoices/${matchId}/confirm`,
        payload,
      );
      return data;
    },

    onSuccess: (response) => {
      // Patch the detail cache in place rather than refetching - the user is
      // about to navigate away, so a refetch would be wasted.
      queryClient.setQueryData<MatchDetail>(
        queryKeys.invoices.detail(matchId),
        (prev) => (prev ? { ...prev, ...response.match } : prev),
      );

      void queryClient.invalidateQueries({
        queryKey: queryKeys.invoices.lists(),
      });

      // A learned alias changes future matching, so the KB list is now stale.
      if (response.created_alias) {
        void queryClient.invalidateQueries({
          queryKey: queryKeys.knowledgeBase.all(),
        });
      }

      toast.success(
        response.odoo_bill_name
          ? `Vendor bill ${response.odoo_bill_name} created in Odoo`
          : 'Match confirmed',
      );
    },
  });
}

/* =====
 * Async PO search - debounced
 * ===== */

export function useSearchPurchaseOrders(rawQuery: string, limit = 20) {
  const query = useDebounceValue(rawQuery.trim(), 300);

  const result = useQuery({
    queryKey: queryKeys.purchaseOrders.search(query),
    queryFn: async ({ signal }) => {
      const { data } = await apiClient.get<POCandidate[]>(
        '/purchase-orders/search',
        { params: { q: query, limit }, signal },
      );
      return data;
    },
    // Don't hammer Odoo on every keystroke, or on a one-character query.
    enabled: query.length >= 2,
    staleTime: 30_000,
    placeholderData: keepPreviousData,
  });

  return {
    ...result,
    // The combobox needs to distinguish "typing settled, still loading" from

```

```
// "waiting for the debounce" to avoid a flashing spinner.
isDebouncing: rawQuery.trim() !== query,
};
}
```

Verification screen

Choosing the PDF viewer

	<iframe> (native viewer)	react-pdf (pdf.js)
Dependencies	0	~1.5 MB (lazy-loaded worker)
Zoom / page nav	Browser chrome only — not programmatically controllable	Full control
Consistent cross-browser UI	No	Yes
Renders image invoices	No — needs a separate branch anyway	No — same branch needed
Overlay OCR bounding boxes	Impossible	Straightforward
Setup complexity	Trivial	Worker config + <code>ssr: false</code>

Pick `react-pdf`. Zoom and page navigation are explicit requirements and an `iframe` cannot provide them programmatically. More importantly, the natural next feature here is *click a field in the comparison table, highlight where it was found on the page* — that requires pdf.js. Choosing `iframe` now guarantees a rewrite later. Keep an `<iframe>` as an error-boundary fallback only.

```
// src/components/features/verification/pdf-viewer.tsx
'use client';

import { useState } from 'react';
import { Document, Page, pdfjs } from 'react-pdf';
import { Loader2, FileWarning } from 'lucide-react';
import 'react-pdf/dist/Page/AnnotationLayer.css';
import 'react-pdf/dist/Page/TextLayer.css';

/**
 * MUST be set in the same module that renders <Document/>, or react-pdf's
 * default workerSrc wins the race and overwrites it.
 *
 * pdfjs-dist is pinned to EXACTLY the version react-pdf depends on (5.4.296).
 * If it drifts, pdf.js throws:
 * "The API version X does not match the Worker version Y"
 */
pdfjs.GlobalWorkerOptions.workerSrc = new URL(
  'pdfjs-dist/build/pdf.worker.min.mjs',
  import.meta.url,
).toString();

export interface PdfViewerProps {
  /** Same-origin URL; the httpOnly cookie authenticates it automatically. */
  fileUrl: string;
  pageNumber: number;
  zoom: number;
  rotation: number;
  onLoadSuccess: (pageCount: number) => void;
}

export function PdfViewer({
  fileUrl, pageNumber, zoom, rotation, onLoadSuccess,
```

```

}: PdfViewerProps) {
  const [error, setError] = useState<Error | null>(null);

  if (error) {
    return (
      <div className="flex h-full flex-col items-center justify-center gap-3 p-6 text-center">
        <FileWarning className="text-muted-foreground size-8" />
        <p className="text-muted-foreground text-sm">
          Could not render this PDF inline.
        </p>
        <a
          href={fileUrl}
          target="_blank"
          rel="noreferrer"
          className="text-primary text-sm underline"
        >
          Open in a new tab
        </a>
      </div>
    );
  }

  return (
    <Document
      file={fileUrl}
      onLoadSuccess={({ numPages }) => onLoadSuccess(numPages)}
      onLoadError={setError}
      loading={
        <div className="flex h-full items-center justify-center">
          <Loader2 className="text-muted-foreground size-6 animate-spin" />
        </div>
      }
      className="flex justify-center"
    >
      <Page
        pageNumber={pageNumber}
        scale={zoom}
        rotate={rotation}
        renderAnnotationLayer
        renderTextLayer
        className="shadow-lg"
      />
    </Document>
  );
}

```

```

// src/components/features/verification/invoice-preview.tsx
'use client';

import { useState } from 'react';
import dynamic from 'next/dynamic';
import { Skeleton } from '@components/ui/skeleton';
import { ViewerControls } from './viewer-controls';
import { ImageViewer } from './image-viewer';

/**
 * ssr:false is mandatory - pdf.js touches DOMMatrix/canvas at module scope and
 * crashes Node during SSR. Note this dynamic() call lives inside a Client
 * Component; ssr:false is not permitted from a Server Component.
 */
const PdfViewer = dynamic(
  () => import('./pdf-viewer').then((m) => m.PdfViewer),
  { ssr: false, loading: () => <Skeleton className="h-full w-full" /> },
);

const MIN_ZOOM = 0.5;
const MAX_ZOOM = 3;
const ZOOM_STEP = 0.25;

export interface InvoicePreviewProps {
  fileUrl: string;
  mimeType: string;
  fileName: string;
}

```

```

export function InvoicePreview({
  fileUrl, mimeType, fileName,
}: InvoicePreviewProps) {
  const [pageNumber, setPageNumber] = useState(1);
  const [pageCount, setPageCount] = useState(1);
  const [zoom, setZoom] = useState(1);
  const [rotation, setRotation] = useState(0);

  const isPdf = mimeType === 'application/pdf';

  return (
    <div className="bg-muted/40 flex h-full flex-col">
      <ViewerControls
        fileName={fileName}
        pageNumber={pageNumber}
        pageCount={isPdf ? pageCount : 1}
        zoom={zoom}
        canPaginate={isPdf}
        onPageChange={setPageNumber}
        onZoomIn={() => setZoom((z) => Math.min(MAX_ZOOM, z + ZOOM_STEP))}
        onZoomOut={() => setZoom((z) => Math.max(MIN_ZOOM, z - ZOOM_STEP))}
        onZoomReset={() => setZoom(1)}
        onRotate={() => setRotation((r) => (r + 90) % 360)}
        downloadUrl={fileUrl}
      />

      <div className="flex-1 overflow-auto p-4">
        {isPdf ? (
          <PdfViewer
            fileUrl={fileUrl}
            pageNumber={pageNumber}
            zoom={zoom}
            rotation={rotation}
            onLoadSuccess={setPageCount}
          />
        ) : (
          <ImageViewer
            src={fileUrl}
            alt={fileName}
            zoom={zoom}
            rotation={rotation}
          />
        )}
      </div>
    </div>
  );
}

```

ConfidenceScoreBadge

```

// src/lib/confidence.ts
import type { ConfidenceTier } from '@types/invoice';

export const CONFIDENCE_THRESHOLDS = { high: 90, medium: 70 } as const;

export function getConfidenceTier(score: number): ConfidenceTier {
  if (score >= CONFIDENCE_THRESHOLDS.high) return 'high';
  if (score >= CONFIDENCE_THRESHOLDS.medium) return 'medium';
  return 'low';
}

export const CONFIDENCE_LABEL: Record<ConfidenceTier, string> = {
  high: 'High confidence',
  medium: 'Needs review',
  low: 'Low confidence',
};

export const CONFIDENCE_HINT: Record<ConfidenceTier, string> = {
  high: 'Fields align closely with the matched purchase order. Safe to confirm.',
  medium: 'Some fields disagree. Review the highlighted rows before confirming.',
  low: 'Weak match. Verify every field or select a different purchase order.',
};

```



```
// src/components/shared/confidence-score-badge.tsx
'use client';

import { cva, type VariantProps } from 'class-variance-authority';
import { CheckCircle2, AlertTriangle, XCircle } from 'lucide-react';
import {
  Tooltip, TooltipContent, TooltipTrigger,
} from '@components/ui/tooltip';
import { cn } from '@lib/utils';
import {
  getConfidenceTier, CONFIDENCE_LABEL, CONFIDENCE_HINT,
} from '@lib/confidence';
import type { ConfidenceTier } from '@types/invoice';

const badgeVariants = cva(
  'inline-flex items-center gap-1.5 rounded-full border font-medium tabular-nums transition-colors',
  {
    variants: {
      tier: {
        // Explicit light/dark pairs - semantic colors must stay legible in both.
        high: 'border-emerald-600/20 bg-emerald-50 text-emerald-700 dark:border-emerald-400/20 dark:bg-emerald-950 dark:text-emerald-300',
        medium: 'border-amber-600/20 bg-amber-50 text-amber-700 dark:border-amber-400/20 dark:bg-amber-950 dark:text-amber-300',
        low: 'border-red-600/20 bg-red-50 text-red-700 dark:border-red-400/20 dark:bg-red-950 dark:text-red-300',
      },
      size: {
        sm: 'px-2 py-0.5 text-xs',
        md: 'px-2.5 py-1 text-sm',
        lg: 'px-3.5 py-1.5 text-base',
      },
    },
    defaultVariants: { tier: 'low', size: 'md' },
  },
);

const TIER_ICON: Record<ConfidenceTier, typeof CheckCircle2> = {
  high: CheckCircle2,
  medium: AlertTriangle,
  low: XCircle,
};

const ICON_SIZE = { sm: 'size-3', md: 'size-3.5', lg: 'size-4' } as const;

export interface ConfidenceScoreBadgeProps
  extends Omit<VariantProps<typeof badgeVariants>, 'tier'> {
  /** 0-100. Values outside the range are clamped. */
  score: number;
  tier?: ConfidenceTier;
  showLabel?: boolean;
  showTooltip?: boolean;
  className?: string;
}

export function ConfidenceScoreBadge({
  score, tier, size = 'md',
  showLabel = false, showTooltip = true, className,
}: ConfidenceScoreBadgeProps) {
  const safeScore = Math.max(0, Math.min(100, Math.round(score)));
  const resolvedTier = tier ?? getConfidenceTier(safeScore);
  const Icon = TIER_ICON[resolvedTier];

  const badge = (
    <span
      className={cn(badgeVariants({ tier: resolvedTier, size }), className)}
      // Screen readers get the meaning, not just a bare number.
      aria-label={` ${CONFIDENCE_LABEL[resolvedTier]}: ${safeScore} out of 100`}
    >
      <Icon className={ICON_SIZE[size ?? 'md']} aria-hidden="true" />
      <span>{safeScore}</span>
      {showLabel && (
        <span className="font-normal">{CONFIDENCE_LABEL[resolvedTier]}</span>
      )}
    </span>
  );
}
```

```

if (!showTooltip) return badge;

return (
  <Tooltip>
    <TooltipTrigger asChild>{badge}</TooltipTrigger>
    <TooltipContent side="bottom" className="max-w-64">
      <p className="font-medium">{CONFIDENCE_LABEL[resolvedTier]}</p>
      <p className="text-muted-foreground text-xs">
        {CONFIDENCE_HINT[resolvedTier]}
      </p>
    </TooltipContent>
  </Tooltip>
);
}

```

Field comparison

```

// src/lib/compare.ts
import type { FieldMatchState } from '@types/invoice';

/** Loose equality for OCR output: case/whitespace/punctuation insensitive. */
export function normalizeText(value: string): string {
  return value.toLowerCase().replace(/[^a-z0-9]/g, '');
}

export function compareText(
  a: string | null, b: string | null,
): FieldMatchState {
  if (!a || !b) return 'missing';
  const na = normalizeText(a);
  const nb = normalizeText(b);
  if (na === nb) return 'match';
  // Containment covers "ACME Corp" vs "ACME Corporation Ltd".
  if (na.length > 2 && nb.length > 2 && (na.includes(nb) || nb.includes(na))) {
    return 'partial';
  }
  return 'mismatch';
}

/** Money compares with a tolerance - OCR rounding and tax deltas are normal. */
export function compareAmount(
  a: number | null, b: number | null, tolerance = 0.01,
): FieldMatchState {
  if (a === null || b === null) return 'missing';
  const diff = Math.abs(a - b);
  if (diff <= tolerance) return 'match';
  // Within 2% is likely a tax or rounding discrepancy, not the wrong PO.
  if (b !== 0 && diff / Math.abs(b) <= 0.02) return 'partial';
  return 'mismatch';
}

export function compareDate(
  a: string | null, b: string | null, dayTolerance = 0,
): FieldMatchState {
  if (!a || !b) return 'missing';
  const da = new Date(a).getTime();
  const db = new Date(b).getTime();
  if (Number.isNaN(da) || Number.isNaN(db)) return 'missing';
  const days = Math.abs(da - db) / 86_400_000;
  if (days <= dayTolerance) return 'match';
  if (days <= 30) return 'partial';
  return 'mismatch';
}

```

```

// src/components/features/verification/comparison-table.tsx
'use client';

import { Check, X, AlertCircle, Minus, Pencil } from 'lucide-react';
import {
  Table, TableBody, TableCell, TableHead, TableHeader, TableRow,
} from '@components/ui/table';
import { Button } from '@components/ui/button';

```

```

import {
  Tooltip, TooltipContent, TooltipTrigger,
} from '@components/ui/tooltip';
import { cn } from '@lib/utils';
import type { FieldMatchState } from '@types/invoice';

export interface ComparisonField {
  key: string;
  label: string;
  /** Pre-formatted for display; the caller does currency/date formatting. */
  invoiceValue: string | null;
  poValue: string | null;
  state: FieldMatchState;
  editable?: boolean;
  /** True when the user has overridden the OCR value. */
  corrected?: boolean;
}

export interface ComparisonTableProps {
  fields: ComparisonField[];
  poLabel?: string;
  onEditField?: (key: string) => void;
  className?: string;
}

const STATE_CONFIG: Record<
  FieldMatchState,
  { Icon: typeof Check; className: string; label: string; rowClass: string }
> = {
  match: {
    Icon: Check,
    className: 'text-emerald-600 dark:text-emerald-400',
    label: 'Values match',
    rowClass: '',
  },
  partial: {
    Icon: AlertCircle,
    className: 'text-amber-600 dark:text-amber-400',
    label: 'Close, but not identical',
    rowClass: 'bg-amber-50/50 dark:bg-amber-950/20',
  },
  mismatch: {
    Icon: X,
    className: 'text-red-600 dark:text-red-400',
    label: 'Values do not match',
    rowClass: 'bg-red-50/50 dark:bg-red-950/20',
  },
  missing: {
    Icon: Minus,
    className: 'text-muted-foreground',
    label: 'Value missing on one side',
    rowClass: '',
  },
};

export function ComparisonTable({
  fields, poLabel = 'Purchase Order', onEditField, className,
}: ComparisonTableProps) {
  return (
    <div className={cn('rounded-md border', className)}>
      <Table>
        <TableHeader>
          <TableRow className="hover:bg-transparent">
            <TableHead className="w-[26%]">Field</TableHead>
            <TableHead className="w-[32%]">Extracted (OCR)</TableHead>
            <TableHead className="w-10 text-center">
              <span className="sr-only">Match status</span>
            </TableHead>
            <TableHead className="w-[32%]">{poLabel}</TableHead>
            {onEditField && (
              <TableHead className="w-10">
                <span className="sr-only">Edit</span>
              </TableHead>
            )}
          </TableRow>
        </TableHeader>

```

```

<TableBody>
  {fields.map((field) => {
    const { Icon, className: iconClass, label, rowClass } =
      STATE_CONFIG[field.state];

    return (
      <TableRow key={field.key} className={rowClass}>
        <TableCell className="text-muted-foreground font-medium">
          {field.label}
        </TableCell>

        <TableCell className="font-mono text-sm">
          <span
            className={cn(
              field.corrected && 'text-primary font-semibold',
            )}
          >
            {field.invoiceValue ?? (
              <span className="text-muted-foreground italic">✖</span>
            )}
          </span>
          {field.corrected && (
            <span className="text-primary ml-2 text-xs font-normal">
              (edited)
            </span>
          )}
        </TableCell>

        <TableCell className="text-center">
          <Tooltip>
            <TooltipTrigger asChild>
              <span className="inline-flex">
                <Icon
                  className={cn('size-4', iconClass)}
                  aria-label={label}
                />
              </span>
            </TooltipTrigger>
            <TooltipContent>{label}</TooltipContent>
          </Tooltip>
        </TableCell>

        <TableCell className="font-mono text-sm">
          {field.poValue ?? (
            <span className="text-muted-foreground italic">✖</span>
          )}
        </TableCell>

        {onEditField && (
          <TableCell>
            {field.editable && (
              <Button
                variant="ghost"
                size="icon"
                className="size-7"
                onClick={() => onEditField(field.key)}
                aria-label={`Edit ${field.label}`}
              >
                <Pencil className="size-3.5" />
              </Button>
            )}
          </TableCell>
        )}
      </TableRow>
    );
  })}
</TableBody>
</Table>
</div>
);
}

```

Remaining components

Component	Props	Responsibility
VerificationScreen	<code>{ matchId: string }</code>	Top-level client owner. Holds <code>selectedPoId</code> , <code>corrections</code> , <code>isDirty</code> . Reads <code>useMatchDetail</code> , derives comparison fields via <code>lib/compare.ts</code> , lays out the resizable panels. The only component that calls <code>useConfirmMatch</code> .
VerificationToolbar	<code>{ match, isDirty, isSubmitting, onConfirm, onReject }</code>	Sticky header: filename, <code>StatusBadge</code> , <code>ConfidenceScoreBadge</code> (size <code>lg</code>), Confirm/Reject. Confirm disabled while <code>selectedPoId === null</code> .
ViewerControls	<code>{ fileName, pageNumber, pageCount, zoom, canPaginate, onPageChange, onZoomIn, onZoomOut, onZoomReset, onRotate, downloadUrl }</code>	Presentational toolbar. Disables prev/next at bounds; shows <code>Math.round(zoom*100)%</code> .
ImageViewer	<code>{ src, alt, zoom, rotation }</code>	Renders JPG/PNG invoices with the same zoom/rotate contract as <code>PdfViewer</code> . Uses a plain <code></code> , not <code>next/image</code> — the source is an authenticated dynamic byte stream, so the optimizer is useless.
LineItemsComparison	<code>{ ocrLines, poLines, currency }</code>	Greedy-aligns OCR lines to PO lines by SKU, then description similarity, then index. Paired rows with per-cell qty/price/total deltas; unmatched lines in an "Unmatched" group with an amber rail; totals reconciliation footer.
POSelectorCombobox	<code>{ value, candidates, onChange, disabled? }</code>	shadcn <code>Popover</code> + <code>Command</code> with <code>shouldFilter={false}</code> (the server filters). Shows ranked auto-match candidates when empty, switches to <code>useSearchPurchaseOrders</code> at ≥ 2 chars. Each row: PO name, partner, date, total, score badge.
ScoreBreakdownPanel	<code>{ breakdown, onFieldClick? }</code>	Per-component rows: label, weight %, a <code>Progress</code> bar tinted by tier, <code>weighted_score</code> , <code>rationale</code> in muted text. Shows an "alias applied" note when <code>alias_applied</code> . Clicking a row scrolls the matching <code>ComparisonTable</code> row into view.
CandidateList	<code>{ candidates, selectedPoId, onSelect }</code>	Ranked radio-style cards for the top N, so the user can flip between candidates without opening the combobox.
CorrectionForm	<code>{ invoice, onChange, disabled? }</code>	RHF + <code>zodResolver</code> , <code>mode: 'onBlur'</code> . Watches the form and pushes a diff against the original upward — never the whole object — so the backend only learns real corrections.
ConfirmMatchDialog	<code>{ open, onOpenChange, match, selectedPo, corrections, isSubmitting, onConfirm }</code>	Final review: summary of edits, <code>learn_vendor_alias</code> switch (default on when <code>vendor_name</code> was corrected), <code>push_to_odoo</code> switch, optional notes. Prevents the accidental irreversible Odoo write.

```
// src/components/features/verification/verification-schemas.ts
import { z } from 'zod';

// Zod 4 idioms: top-level format fns (z.iso.date()) and `error` instead of
// `message`. `message` still works but is deprecated.
const optionalMoney = z
  .number({ error: 'Must be a number' })
  .nonnegative({ error: 'Cannot be negative' })
  .nullable();

export const correctionSchema = z
  .object({
    vendor_name: z.string().trim().min(1, { error: 'Vendor is required' }).nullable(),
    invoice_number: z.string().trim().min(1, { error: 'Invoice number is required' }).nullable(),
    invoice_date: z.iso.date({ error: 'Use YYYY-MM-DD' }).nullable(),
    due_date: z.iso.date({ error: 'Use YYYY-MM-DD' }).nullable(),
    po_number: z.string().trim().nullable(),
    currency: z.string().length(3, { error: 'Use a 3-letter ISO code' }).nullable(),
    subtotal: optionalMoney,
    tax_amount: optionalMoney,
    total_amount: optionalMoney,
  })
  .refine(
    (v) =>
      v.subtotal === null ||
      v.tax_amount === null ||
      v.total_amount === null ||
      Math.abs(v.subtotal + v.tax_amount - v.total_amount) < 0.02,
    { error: 'Subtotal + tax must equal the total', path: ['total_amount'] },
  );

export type CorrectionFormValues = z.infer<typeof correctionSchema>;
```

Screen layout

shadcn's `resizable` wraps `react-resizable-panels` v4, whose API uses `orientation`, not the older `direction`. Getting this wrong is a silent no-op.

```
// src/components/features/verification/verification-screen.tsx (abridged)
'use client';

<div className="flex h-[calc(100vh-4rem)] flex-col">
  <VerificationToolbar {...toolbarProps} />

  <ResizablePanelGroup
    orientation="horizontal"
    // Persists the user's split across sessions. AP clerks live on this screen.
    autoSaveId="verification-split"
    className="flex-1"
  >
    <ResizablePanel defaultSize={50} minSize={30}>
      <InvoicePreview
        fileUrl={match.file_url}
        mimeType={match.mime_type}
        fileName={match.file_name}
      />
    </ResizablePanel>

    <ResizableHandle withHandle />

    <ResizablePanel defaultSize={50} minSize={30}>
      <ScrollArea className="h-full">
        <div className="space-y-6 p-6">
          <POSelectorCombobox
            value={selectedPoId}
            candidates={match.candidates}
            onChange={handleSelectPo}
          />

          <Tabs defaultValue="fields">
            <TabsList>
              <TabsTrigger value="fields">Fields</TabsTrigger>
            </TabsList>
          </Tabs>
        </div>
      </ScrollArea>
    </ResizablePanel>
  </ResizablePanelGroup>
</div>
```

```

<TabsTrigger value="lines">
  Line items
  <Badge variant="secondary" className="ml-1.5">
    {match.extracted_invoice.line_items.length}
  </Badge>
</TabsTrigger>
<TabsTrigger value="score">Score</TabsTrigger>
</TabsList>

<TabsContent value="fields">
  <ComparisonTable fields={comparisonFields} onEditField={focusField} />
  <CorrectionForm
    invoice={match.extracted_invoice}
    onChange={setCorrections}
  />
</TabsContent>

<TabsContent value="lines">
  <LineItemsComparison
    ocrLines={match.extracted_invoice.line_items}
    poLines={selectedPo?.lines ?? []}
    currency={match.currency ?? 'USD'}
  />
</TabsContent>

<TabsContent value="score">
  {selectedCandidate && (
    <ScoreBreakdownPanel breakdown={selectedCandidate.score} />
  )}
</TabsContent>
</Tabs>
</div>
</ScrollArea>
</ResizablePanel>
</ResizablePanelGroup>
</div>

```

Below 1024px, render `<Tabs>` (Document | Compare) instead of the panel group — a 50/50 split is unusable on a laptop half-screen.

Server vs Client components

Rule: Server Components own the shell and the *first* payload; TanStack Query owns everything that changes.

Layer	Kind	Rationale
<code>app/layout.tsx</code> , route-group layouts	RSC	Fonts, metadata, provider mounting. Zero JS.
<code>(dashboard)/layout.tsx</code>	RSC	<code>await</code> the user from <code>/auth/me</code> server-side. No auth flicker, no client waterfall.
<code>dashboard/page.tsx</code> , <code>verify/[matchId]/page.tsx</code>	RSC	Thin shells that prefetch + dehydrate , then render a client child inside <code><HydrationBoundary></code> .
<code>loading.tsx</code> , <code>not-found.tsx</code>	RSC	Streamed skeletons.
<code>error.tsx</code>	'use client'	React requires error boundaries to be client components.
Every <code>components/features/*</code> leaf	'use client'	All are interactive.

The payoff pattern — the verification page ships fully-rendered HTML with no client fetch waterfall, yet stays fully interactive and refetchable:

```
// src/app/(dashboard)/verify/[matchId]/page.tsx - RSC
import { QueryClient, dehydrate, HydrationBoundary } from '@tanstack/react-query';
import { notFound } from 'next/navigation';
import { serverFetch } from '@lib/api-server';
import { queryKeys } from '@lib/query-keys';
import { VerificationScreen } from '@components/features/verification/verification-screen';
import type { MatchDetail } from '@types/invoice';

// Next 16: params is a Promise. PageProps comes from `next typegen`.
export default async function VerifyPage({ params }: PageProps<'verify/[matchId]')> {
  const { matchId } = await params;

  const queryClient = new QueryClient();

  try {
    await queryClient.prefetchQuery({
      queryKey: queryKeys.invoices.detail(matchId),
      queryFn: () => serverFetch<MatchDetail>(`/invoices/${matchId}`),
    });
  } catch {
    notFound();
  }

  return (
    <HydrationBoundary state={dehydrate(queryClient)}>
      <VerificationScreen matchId={matchId} />
    </HydrationBoundary>
  );
}
```

```
// src/lib/api-server.ts
import { cookies } from 'next/headers';

const API_URL = process.env.API_BASE_URL!;

/**
 * Server-side fetch. The browser's httpOnly cookie is NOT automatically attached
 * to server-originated requests, so it must be forwarded explicitly.
 * cookies() is async in Next 16 - the sync form was removed.
 */
export async function serverFetch<T>({
  path: string, init?: RequestInit,
}): Promise<T> {
  const cookieStore = await cookies();
  const token = cookieStore.get('access_token')?.value;

  const response = await fetch(`${API_URL}${path}`, {
    ...init,
    headers: {
      ...init?.headers,
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      Accept: 'application/json',
    },
    // Per-user authenticated data must never enter the shared route cache.
    cache: 'no-store',
  });

  if (!response.ok) throw new Error(`API ${response.status} on ${path}`);
  return response.json() as Promise<T>;
}
```

Server Actions are deliberately unused. The core mutations — upload and confirm — need upload progress and fine-grained cache updates, which Server Actions do not provide. Mixing both would mean two competing cache systems. Keep all mutations in TanStack Query and use RSC only for the initial read.

Configuration

.env.example

```
# ---- Server-only (never exposed to the browser) ----
API_BASE_URL=http://localhost:8000/api/v1

# ---- Public (inlined into the client bundle at build time) ----
# Same-origin path so the rewrite handles proxying.
NEXT_PUBLIC_API_BASE_URL=/api/v1

# 'cookie' (recommended) | 'localStorage' (fallback)
NEXT_PUBLIC_AUTH_MODE=cookie

NEXT_PUBLIC_MAX_UPLOAD_MB=20
```

`serverRuntimeConfig` / `publicRuntimeConfig` were removed in Next 16. Plain env vars are the only option. For a value read at runtime rather than baked at build time, call `await connection()` before reading `process.env`.

next.config.ts

```
import type { NextConfig } from 'next';

const API_BASE_URL =
  process.env.API_BASE_URL ?? 'http://localhost:8000/api/v1';

const nextConfig: NextConfig = {
  reactStrictMode: true,
  typescript: { ignoreBuildErrors: false },

  // NOTE: the `eslint` config key was REMOVED in Next 16, and `next build` no
  // longer lints. Run `eslint .` as a separate CI step.

  /**
   * Rewrites, not CORS:
   * - The browser only ever talks to its own origin -> zero preflight requests.
   * - The httpOnly auth cookie is same-origin, so it is sent automatically,
   *   including on <Document file="/api/v1/invoices/x/file" /> in the PDF
   *   viewer, which would otherwise need a manual authenticated blob fetch.
   * - No FastAPI CORSMiddleware to misconfigure in production.
   *
   * Cost: one extra network hop in dev. In production, put Next and FastAPI
   * behind the same ingress and it is free.
   */
  async rewrites() {
    return [
      { source: '/api/v1/:path*', destination: `${API_BASE_URL}/:path*` },
    ];
  },

  async headers() {
    return [
      {
        source: '/:path*',
        headers: [
          { key: 'X-Content-Type-Options', value: 'nosniff' },
          { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
          { key: 'X-Frame-Options', value: 'DENY' },
        ],
      },
    ];
  },
};

export default nextConfig;
```

Rewrites vs direct CORS — pick rewrites. Direct CORS calls add a preflight to every request, force `SameSite=None; Secure` cookies (which Safari's ITP degrades), break the PDF viewer's simple `file={url}` usage, and require an allow-list per environment. The only advantage — skipping one hop — disappears when both services sit behind one ingress.

Upload caveat to verify in staging. The multipart upload streams through Next's rewrite proxy. Confirm your platform's request body limit accommodates `NEXT_PUBLIC_MAX_UPLOAD_MB` — Vercel's serverless functions cap at ~4.5 MB. If you deploy there and need larger invoices, either self-host Next in a container or have the browser POST directly to FastAPI for that one route.

shadcn/ui setup

```
npx shadcn@latest init -t next
```

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "new-york",
  "rsc": true,
  "tsx": true,
  "tailwind": {
    "config": "",
    "css": "src/app/globals.css",
    "baseColor": "slate",
    "cssVariables": true,
    "prefix": ""
  },
  "iconLibrary": "lucide",
  "aliases": {
    "components": "@components",
    "ui": "@components/ui",
    "utils": "@lib/utils",
    "lib": "@lib",
    "hooks": "@hooks"
  }
}
```

Tailwind v4 has no `tailwind.config.ts` — `tailwind.config` is intentionally `""`. Theme tokens live in `globals.css` under `@import "tailwindcss";` plus `@theme inline {...}`. `postcss.config.mjs` contains only `{ plugins: { '@tailwindcss/postcss': {} } }`.

```
npx shadcn@latest add \
  alert alert-dialog avatar badge breadcrumb button card checkbox \
  command dialog dropdown-menu form input label pagination popover \
  progress resizable scroll-area select separator sheet skeleton \
  sonner switch table tabs textarea tooltip
```

Mapping to screens: `command` + `popover` → `P0SelectorCombobox`; `resizable` + `scroll-area` → verification layout; `form` + `input` + `label` + `select` + `textarea` → `CorrectionForm`; `progress` → upload and `ScoreBreakdownPanel`; `table` → history and comparisons; `alert-dialog` → destructive reject; `sonner` → the `<Toaster/>` in `app/layout.tsx`.

Implementation sequence

1. **Scaffold & config** — `create-next-app`, pin `package.json`, shadcn init, flat ESLint config, `next.config.ts`, `.env`. Gate: `build`, `lint` and `typecheck` all pass clean.
2. **Agree the API contract** (document 04) with the backend. Everything downstream depends on it.
3. **Types + lib primitives** — `types/*`, `lib/*`, `providers/*`. No UI yet; pure and unit-testable.
4. **Auth vertical slice** — Route Handlers, `proxy.ts`, `(auth)` pages, `use-auth`. Gate: `login` sets the cookie, `/dashboard` redirects when signed out, and a forced 401 triggers exactly one refresh.
5. **Dashboard** — table, filters, pagination, RSC prefetch. Proves the RSC/Query hydration pattern end to end.
6. **Upload** — dropzone, progress, cancel, redirect to `/verify/[id]`.
7. **Verification screen** — in this order: `InvoicePreview` (highest technical risk — the worker config) → `ComparisonTable` → `ScoreBreakdownPanel` → `LineItemsComparison` → `POSelectorCombobox` → `CorrectionForm` → `ConfirmMatchDialog`.
8. **Knowledge base admin** — simplest CRUD, safe to leave last.
9. **Hardening** — accessibility pass on the comparison tables, the `<1024px` tab fallback, error boundaries, empty states, focus management in the combobox.

Risks, ranked

1. **pdf.js worker/version skew** — mitigated by the exact `5.4.296` pin plus `overrides`. Verify on day one of step 7; the symptom is the API/Worker version error, not a blank screen.
2. **Backend contract drift** — greenfield on both sides. Once FastAPI runs, generate types from `/openapi.json` so drift becomes a compile error.
3. **Long OCR requests** — a 120s axios timeout plus an ingress defaulting to 60s produces confusing 504s. Raise the ingress timeout, or move upload to 202 + poll — `useMatchDetail` already has the polling logic.
4. **Line-item alignment quality** — the greedy matcher is heuristic. Keep it in `lib/compare.ts` (pure, unit-tested) so it can be swapped for backend-provided alignment without touching components.
5. **TypeScript 7 pressure** — the ecosystem will migrate over coming quarters. Revisit the `6.0.3` pin once `typescript-eslint` widens its peer range past `<6.1.0`.